Structure-Aware Landmark Selection Strategy for Fast Shortest Path Distance Estimation in Large Networks

Social Network Analysis for Computer Scientists — Course paper

Godwin Addetsi
g.addetsi@umail.leidenuniv.nl
LIACS, Leiden University

Nallathambi Vethiappan
n.vethiappan@umail.leidenuniv.nl
LIACS, Leiden University

Abstract

This paper studies shortest-path distance estimation in five real-world graphs, where computing exact distances with Breadth-First Search (BFS) is too expensive. We use a landmark-based approach in which distances from a small set of landmark nodes to all other nodes are precomputed and then used to estimate distances between query node pairs via triangle-inequality bounds. The accuracy of this method depends strongly on the landmark choice, but selecting an optimal landmark set is NP-hard, so practical systems rely on heuristics. We compare baseline strategies based on random selection, node degree, and closeness centrality, and propose a structure-aware strategy that selects landmarks using a Brandes-inspired shortest-path participation signal. Our experiments show that random selection is consistently weakest, and that the Brandes-inspired strategy is most useful when the landmark budget is small, where it can match or improve upon the baselines. Precomputation time grows roughly linearly with the number of landmarks and is dominated by the BFS computations needed to build the landmark distance table.

Keywords

Graphs, shortest path, landmark, real-world networks, social network analysis

ACM Reference Format:
Godwin Addetsi and Nallathambi Vethiappan. 2025. Structure-Aware Landmark Selection Strategy for Fast Shortest Path Distance Estimation in Large Networks: Social Network Analysis for Computer Scientists — Course paper. In *Proceedings of Social Network Analysis for Computer Scientists Course 2025 (SNACS '25)*. ACM, New York, NY, USA, 8 pages.

1 Introduction

Efficient computation of shortest-path distances in large-scale networks is a fundamental problem in computer science, with applications spanning social network analysis, communication routing, biological networks, and information retrieval [6, 14]. In this report, we study the task of answering shortest-path distance queries between node pairs in a large, undirected, unweighted, connected graph.

In unweighted graphs, exact shortest-path distances from a source can be computed with Breadth-First Search (BFS). For a graph with n nodes and m edges, a BFS processes each edge at most once and runs in $O(m)$ time. While this is practical for a small number of queries, it becomes expensive when distances must be computed repeatedly at scale. Running BFS from every source node results in $O(mn)$ total work, which corresponds to solving the All-Pairs Shortest Path (APSP) problem. Storing all-pairs distances is also prohibitive, since a full distance matrix requires $O(n^2)$ space.

These limitations motivate approximation and estimation techniques for shortest-path distances. Among these, landmark-based methods have emerged as a practical solution for large graphs [13]. A small set of landmark nodes D is selected and exact distances from each landmark to every node are precomputed. When a distance query (s, t) arrives, the landmark distances are combined using triangle-inequality bounds to obtain an estimate. Since $|D| \ll n$, the stored information and per-query computation are much smaller than running a fresh BFS for each query [13].

The accuracy of this framework depends strongly on landmark selection. Random landmarks provide a simple baseline, but heuristics such as choosing high-degree nodes or closeness-based landmarks can improve accuracy in practice [13]. Since finding an optimal landmark set is NP-hard [13], practical methods rely on heuristics. A common failure mode of ranking-based heuristics is redundancy, where many top-ranked nodes lie in the same region of the graph and contribute similar information.

We propose a structure-aware landmark selection strategy inspired by Brandes' algorithm [7]. During BFS traversals we compute a Brandes-style dependency score that serves as a shortest-path participation signal, and we select landmarks iteratively based on this score. We additionally evaluate a hop-exclusion variant that avoids selecting landmarks that are too close to previously selected ones, reducing redundancy.

The contributions of this paper are as follows: (i) we introduce and implement a Brandes-inspired frequency-based landmark selection strategy; (ii) we evaluate hop-exclusion with $h_{\min} \in \{0, 1\}$ to control redundancy in the landmark set; and (iii) we empirically compare our method against random, degree-based, and sampled-closeness baselines across multiple landmark budgets on several real-world graphs.

The remainder of this paper is organised as follows. Section 2 reviews prior work on exact shortest-path computation and scalable distance estimation, with emphasis on landmark-based methods. Section 3 introduces the graph notation, recalls the triangle-inequality bounds used for distance estimation, and formalises the distance-query setting; it also summarises the Brandes-style dependency mechanism that we later reuse for landmark selection. Section 4 then presents the methods evaluated in this report, including the baseline landmark selection strategies and our structure-aware,

SNACS '25, Leiden, the Netherlands

Brandes-inspired selection strategy with hop-exclusion. Section 5 describes the datasets, and Section 6 reports the experimental setup and results. Finally, Section 7 summarises the findings and outlines directions for future work.

2 Related work

Computing shortest-path distances exactly has been studied extensively. In unweighted graphs, single-source distances can be obtained with Breadth-First Search (BFS) [8], and in weighted graphs Dijkstra's algorithm is a classical approach [9]. For all-pairs shortest paths, dynamic programming methods such as Floyd–Warshall [10] are well-known, but their time and memory requirements make them impractical on large networks.

Because exact computation does not scale when many distance queries must be answered, a variety of preprocessing-based approximation techniques have been proposed. Distance oracles provide theoretical guarantees by building an index that supports fast approximate queries [15]. Another practical direction is landmark-based distance estimation, where distances from a small set of selected nodes are precomputed and later combined via bounds to answer arbitrary queries quickly [13]. Potamias et al. [13] studied landmark selection strategies and showed that simple heuristics such as choosing high-degree or central nodes can outperform random selection. They also showed that selecting an optimal landmark set is NP-hard [13], which motivates the use of heuristics. Related approaches include combining landmarks with heuristic search (ALT) [11] and embedding-based methods that map graph distances into geometric spaces [12].

Our work builds on the landmark-based setting of [13], and uses the shortest-path dependency accumulation idea behind Brandes' betweenness algorithm [7] as a signal for landmark selection. In contrast to static ranking heuristics such as degree or sampled closeness, we propose a structure-aware strategy that prioritises nodes with high shortest-path participation and optionally enforces hop-based exclusion to reduce redundancy among selected landmarks.

3 Preliminaries

In this section, we introduce the notation used throughout the report. We recall the triangle inequality for shortest-path distance and define the distance estimation setting. We then describe the landmark-based framework and provide background on Brandes' algorithm, which is used later for landmark selection.

3.1 Notation

We consider an undirected and unweighted graph $G = (V, E)$ with $n = |V|$ nodes and $m = |E|$ edges. Because the graph is undirected, $(u, v) \in E$ if and only if $(v, u) \in E$. A path is defined as a sequence of nodes connected by edges. A shortest path between two nodes $u, v \in V$ is a path consisting of a minimal number of edges that connects the two nodes. The length of this shortest path, or the distance $d_G(u, v)$ between nodes u and v, is the number of edges in such a shortest path. We assume that G is connected, i.e., $d_G(u, v)$ is finite for all nodes u and v. The degree $\deg(v)$ of a node v is the number of edges connected to that node.

3.2 Triangle inequality

Shortest-path distance is a metric and therefore satisfies the triangle inequality. For any three nodes $s, u, t \in V$,

$$d_G(s, t) \leq d_G(s, u) + d_G(u, t).$$

Rearranging gives a corresponding lower bound,

$$d_G(s, t) \geq \left| d_G(s, u) - d_G(u, t) \right|.$$

Equivalently,

$$\left| d_G(s, u) - d_G(t, u) \right| \leq d_G(s, t) \leq d_G(s, u) + d_G(t, u).$$

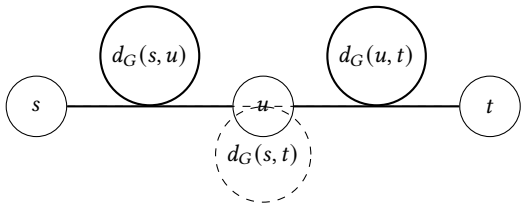


Figure 1: Illustration of the triangle inequality for shortest-path distance.

Figure 1 illustrates that the distance between s and t is bounded by the length of a route that goes via an intermediate node u. These inequalities form the basis for the distance estimation.

3.3 Problem definition

We study the problem of estimating the distance $d_G(u, v)$ between any pair of nodes $u, v \in V$ both accurately and efficiently. Accuracy means that the estimate $\hat{d}(u, v)$ should be close to the true distance. The estimation error should be as small as possible.

Efficiency means that answering one distance query should be much faster than computing an exact distance by BFS. We call the computation time for answering a single query the *query time*. In an unweighted graph, a BFS takes $O(m)$ time. In practice, we want to answer many queries, so the query time should be small. It should not require a graph traversal. In the worst case it should scale at most linearly with the number of nodes, so $O(n)$.

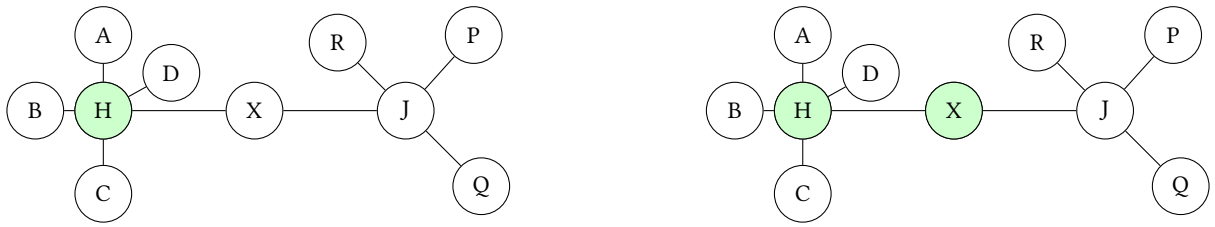
We allow an offline *precomputation* phase that builds an index from the graph before any queries are asked. This precomputation should remain limited. It should be possible to scan the graph only a constant number of times, for example, by performing a small number of full BFS runs. For a small constant $c > 0$, we therefore assume a precomputation budget of $O(c\,m)$.

3.4 Landmarks

As a precomputation step, we select a set of landmark nodes $D \subseteq V$ with $k = |D|$. For each landmark $u \in D$ and each node $v \in V$, we precompute the exact distance $d_G(u, v)$. Since the graph is undirected, this also provides $d_G(v, u)$. These precomputed distances allow us to answer many distance queries without running a graph traversal per query. We discuss strategies for choosing the landmark set D in Section 4.

Using the triangle inequality from Section 3.2, each landmark $u \in D$ induces bounds on the unknown distance $d_G(s, t)$:

$$|d_G(s, u) - d_G(t, u)| \leq d_G(s, t) \leq d_G(s, u) + d_G(t, u).$$



(a) Step 1: choose the highest-degree node as the first landmark. **(b) Step 2: select the next landmark using the participation score.**

Figure 2: Two-step illustration of the structure-aware landmark selection idea. The first landmark is selected by the highest degree. The next landmark is chosen by a Brandes-inspired shortest-path participation score, which tends to prefer bridge nodes. Green nodes indicate selected landmarks.

Taking the tightest bounds over all landmarks yields

$$L(s,t) = \max_{u \in D} \left| d_G(s,u) - d_G(t,u) \right|,$$
$$U(s,t) = \min_{u \in D} \left(d_G(s,u) + d_G(t,u) \right).$$

so that $L(s,t) \le d_G(s,t) \le U(s,t)$.

In this report, we use the upper bound as the distance estimate, i.e., $\hat{d}(s,t) = U(s,t)$. This choice never underestimates the true distance. It becomes exact whenever there exists a landmark $u \in D$ that lies on a shortest path between s and t. In that case $d_G(s,t) = d_G(s,u) + d_G(u,t)$ and the minimum attains the true distance.

3.5 Brandes' algorithm

Betweenness centrality measures how strongly a node participates in shortest paths. Let σ_{st} be the number of shortest paths between nodes s and t, and let $\sigma_{st}(v)$ be the number of those shortest paths that pass through v. The betweenness centrality of v is defined as

$$\mathrm{BC}(v) = \sum_{\substack{s \neq v \neq t \\ s,t \in V}} \frac{\sigma_{st}(v)}{\sigma_{st}}.$$

Brandes' algorithm computes betweenness centrality efficiently by aggregating contributions from each source node s. For an unweighted graph, a BFS from s computes distances, the number of shortest paths from s to each node $\sigma_s(\cdot)$, and predecessor sets $P_s(\cdot)$. A reverse traversal then accumulates dependency scores

$$\delta_s(v) = \sum_{w:\, v \in P_s(w)} \frac{\sigma_s(v)}{\sigma_s(w)} (1 + \delta_s(w)).$$

Summing $\delta_s(v)$ over all sources yields $\mathrm{BC}(v)$ [7].

We use this shortest-path counting and dependency-accumulation mechanism as a building block for landmark selection in Section 4.2.

4 Approach

We use a landmark-based framework to estimate shortest-path distances in large graphs. The method has an offline phase in which we select a landmark set D and precompute distances from each landmark to all nodes, and an online phase in which distance queries are answered using only the stored landmark distances. In the offline phase, we run one BFS per landmark and store all distances from the landmarks to all nodes. For $k = |D|$ landmarks this costs $O(k(n+m))$ time and $O(kn)$ memory. In the online phase, a query is answered in $O(k)$ time by scanning the stored landmark distances.

We evaluate baseline landmark selection strategies and compare them against our structure-aware strategy based on a Brandes-inspired shortest-path participation score.

4.1 Existing landmark selection strategies

In the offline preprocessing phase, we first choose a landmark set D of size k.

We consider three standard baseline strategies for selecting k landmarks:

- **Random:** Select k nodes uniformly at random from V.
- **Degree:** Rank nodes by degree $\deg(v)$ in descending order and select the top k.
- **Closeness:** Approximate closeness centrality by running BFS from a small set of sampled sources and ranking nodes by their average distance to these sources.

Computing exact closeness for all nodes is expensive, since it requires distances from each node to all others and takes $O(mn)$ time in unweighted graphs. We therefore estimate closeness by sampling $c = 200$ source nodes and running BFS from each sample, which reduces the cost to $O(c(n+m))$. We choose $c = 200$ as a practical trade-off between runtime and ranking stability.

4.2 Our contribution: Structure-Aware landmark selection

Our structure-aware strategy aims to select nodes that participate frequently in shortest paths. We start with a seed landmark chosen by highest degree, which is a common heuristic and provides a well-connected initial reference point. For each selected landmark ℓ, we run BFS from ℓ to compute distances and, during the same traversal, compute Brandes-style dependency values. These values act as a signal reflecting how frequently a node lies on shortest paths from the current landmark. We accumulate them into a global participation score $p(v)$. The next landmark is chosen as the highest-scoring node among the allowed candidates, with a fallback to the highest-degree candidate if the scores are uninformative.

Figure 2 illustrates the intuition. In Figure 2(a), H is selected because it has the highest degree in the example graph (degree 4). After selecting H and running BFS from it, the bridge node X receives a high participation score because many shortest paths from H to nodes in the right part of the graph pass through X. Therefore, in Figure 2(b), X is selected as the next landmark. This

procedure is summarised in Algorithm 1. The selection is repeated until the landmark budget k is reached, i.e., until $|D| = k$.

Algorithm 1 Structure-aware landmark selection

Input: Graph $G = (V, E)$, landmark budget k, hop threshold $h_{\min}$
Output: Landmark set D and distance table $Dist[u, \cdot] = d_G(u, \cdot)$ for all $u \in D$

$p(v) \leftarrow 0$ for all $v \in V$; $D \leftarrow \emptyset$; $Dist \leftarrow \emptyset$
$\ell \leftarrow \arg\max_{v \in V} \deg(v)$; $D \leftarrow D \cup \{\ell\}$
while $|D| < k$ **do**
  Run BFS from ℓ and store $Dist[\ell, \cdot] \leftarrow d_G(\ell, \cdot)$
  Compute Brandes-style dependencies $\delta_\ell(\cdot)$
  for each $v \in V \setminus D$ **do** $p(v) \leftarrow p(v) + \delta_\ell(v)$
  end for
  $\ell \leftarrow \arg\max\{\, p(v) \mid v \in V \setminus D,\ \forall u \in D : Dist[u, v] > h_{\min} \,\}$
  if $p(\ell) \leq 0$ **then**
    $\ell \leftarrow \arg\max\{\, \deg(v) \mid v \in V \setminus D,\ \forall u \in D : Dist[u, v] > h_{\min} \,\}$
  end if
  $D \leftarrow D \cup \{\ell\}$
end while
Run BFS from ℓ and store $Dist[\ell, \cdot] \leftarrow d_G(\ell, \cdot)$

4.3 Landmark processing and hop-exclusion

Rank-based strategies can select redundant landmarks that are close together in the graph. To reduce this effect, we also evaluate hop-exclusion during landmark selection. A node is excluded if it lies within $h_{\min}$ hops of any already selected landmark. We test $h_{\min} = 0$ (no exclusion) and $h_{\min} = 1$ (exclude nodes within one hop).

We evaluate each strategy under multiple landmark budgets $|D| \in \{2, 5, 10, 20, 50, 100, 500\}$. For each budget, we select $|D|$ landmarks and build the landmark distance index by running BFS from every selected landmark and storing all distances $d_G(u, v)$ for $u \in D$ and $v \in V$.

4.4 Distance query

Given a query pair (s, t), we compute the estimate using the precomputed landmark distances and return the upper-bound estimator. This computation requires a scan over the k landmarks and does not require a BFS.

5 Data

We use five publicly available real-world networks to study how well landmark-based distance estimation performs in practice. These networks come from social platforms, communication logs, and affiliation data. They differ in size, density, and structure, which gives us a broad setting for comparing the landmark selection methods. Figure 3 illustrates the distance distributions. Next, we provide more details and statistics about the datasets.

Facebook Page-Page network: This dataset contains verified Facebook pages collected through the Facebook Graph API in 2017. Each node is an official page and an edge is present when two pages have liked each other. The dataset contains pages related to companies, television shows, government bodies, and political groups[4].

Douban friendship network: This dataset represents reciprocal friendships on the Douban platform. Nodes represent users and an edge indicates a mutual friendship connection. This network reflects natural social ties formed through shared interests[2].

DBpedia Record Label network: This dataset contains artists and record labels, with an edge showing that an artist has released music under a particular label. This creates a simple two-part structure linking artists to labels [1].

Email-Enron network: This dataset is built from the Enron email corpus. Nodes represent email addresses, and an edge is added when at least one message was exchanged. The dataset includes both internal and external accounts, although only communication involving Enron addresses is fully observed [3].

Twitch Gamers network: This dataset contains users collected from the Twitch public API in 2018. Nodes represent users and an edge shows a mutual following relationship. The graph captures patterns of interaction inside an online gaming community [5].

These datasets are collected from public sources, so some edges or interactions may be missing. This means the graphs may not fully represent the complete underlying networks. In our study, this does not affect the validity of the experiments, because all landmark selection methods are evaluated on exactly the same graphs. Any incompleteness in the data is therefore shared across all methods, and the comparison between them remains fair. All datasets were represented as simple undirected graphs with self-loops removed, using a uniform preprocessing pipeline.

Summary statistics for the datasets are presented in Table 1. For each graph G we report the number of nodes $\|V\|$, the number of edges $\|E\|$, and the average degree $\langle k \rangle$. We also include the average clustering coefficient c with sample size 1000, which reflects how strongly the neighbours of a node tend to be connected with one another. Together, these quantities give a compact view of the connectivity structure of the datasets before applying the landmark selection methods.

Table 1: Summary characteristics of the datasets.

Dataset	$\|V\|$	$\|E\|$	$\langle k \rangle$	c
Facebook Page–Page	22470	170823	15.2045	0.3563
Douban Friendship	154908	327162	4.2240	0.0127
DBpedia Record Label	186689	233286	2.4992	0.0083
Email-Enron	36692	367662	20.0404	0.4888
Twitch Gamers	168114	6797556	80.8684	0.1585

$|V|$: number of nodes, $|E|$: number of edges, $\langle k \rangle$: average degree, c: average clustering coefficient.

6 Experiments

In this section, we run experiments to assess the landmark-based distance estimation methods on the five datasets from Section 5. We compare the three baseline landmark selection strategies from the original work with our Brandes-inspired, structure-aware strategy, and study how the results change with the landmark budget. We first describe the experimental setup and query construction

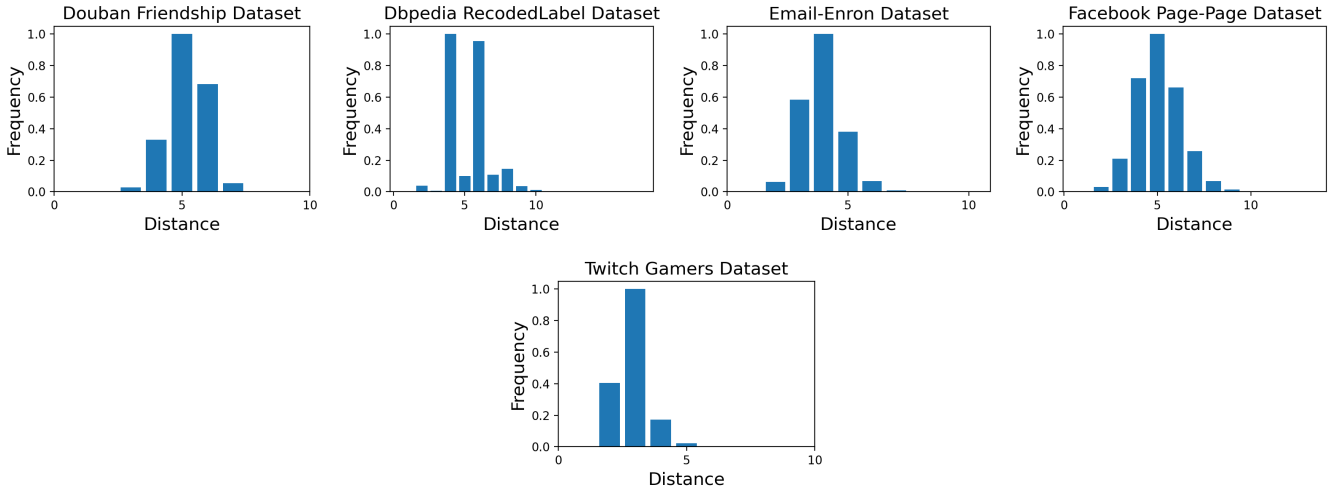


Figure 3: Distributions of distances in our datasets.

in Section 6.1, then define the evaluation metrics in Section 6.2. Finally, we report approximation quality results in Section 6.3 and computational efficiency results in Section 6.4.

6.1 Experimental setup

We report results for landmark budgets $|D| \in \{2, 5, 10, 20, 50, 100, 500\}$. For the strategies that support hop-exclusion (as defined in Section 4.3), we evaluate $h_{\min} \in \{0, 1\}$.

For each dataset, we generate a fixed set of 500 query pairs and reuse it across all strategies and landmark budgets for that dataset. The query set is constructed to cover a range of hop distances: we select random seed nodes, run BFS from each seed, and sample reachable node pairs at different shortest-path lengths. Distances ≤ 2 are treated as short, 3–5 as medium, and ≥ 6 as long. We sample approximately 100 short, 200 medium, and 200 long pairs, and fill any remaining slots with additional random pairs to obtain exactly 500 queries.

All experiments are implemented in Python and executed single-core on an AMD Ryzen 7 7435HS (3.10GHz) machine with 16GB RAM; reported runtimes are wall-clock measurements.

6.2 Evaluation metrics

We evaluate two aspects of the methods: approximation quality and computational efficiency.

Approximation quality is measured by comparing, for each query pair (u, v), the exact shortest-path distance $d_G(u, v)$ with the estimate $\hat{d}(u, v)$. For every query with $d_G(u, v) > 0$ we compute the relative error

$$\varepsilon(u, v) = \frac{|\hat{d}(u, v) - d_G(u, v)|}{d_G(u, v)}.$$

For each dataset, strategy, and landmark budget $|D|$, we report the mean relative error over the 500 evaluation queries.

Computational efficiency is evaluated in both phases. In the offline phase, we record the wall-clock time for landmark selection T_{LM} and for constructing the landmark distance table T_{distance}, and define index construction time as

$$T_{\text{index}} = T_{\text{LM}} + T_{\text{distance}}.$$

In the online phase, we measure the mean wall-clock time per query $T_{\text{query}}(|D|)$ over the same 500 queries for each landmark budget. Since query answering scans the $|D|$ landmark entries, the per-query complexity is $O(|D|)$.

6.3 Approximation quality

Figure 4 reports the mean relative error of the distance estimates as a function of the landmark budget $|D|$ for each dataset. As expected, the error generally decreases as more landmarks are used, because additional landmarks can only tighten the upper-bound estimate $U(s, t)$. Even with very small landmark sets, informed selection is effective. In Douban and Twitch, using only two landmarks chosen by the Brandes-inspired or Degree strategy already yields lower error than using 500 randomly chosen landmarks, reducing the required landmark budget by more than two orders of magnitude. Across all datasets, Random is consistently the weakest baseline.

The differences between the informed strategies are most visible at small budgets. On DBpedia and Email-Enron, Brandes and Degree provide the lowest error for $|D| \in \{2, 5\}$, whereas Sampled Closeness typically improves only after more landmarks are added. On Facebook, the hop-restricted variants of Sampled Closeness and Brandes perform best at very small budgets, suggesting that enforcing a small amount of separation between landmarks can be helpful when only a few landmarks are available. In contrast, on Douban and Twitch the informed strategies behave similarly and their curves converge quickly as $|D|$ increases, indicating that once the landmark budget is moderate the exact selection heuristic matters less.

These observations suggest that the effectiveness of landmark selection depends on multiple interacting structural factors rather than on a single global dataset statistic. While coarse properties such as average degree or clustering coefficient (Table 1) provide some intuition, they do not fully explain the observed differences

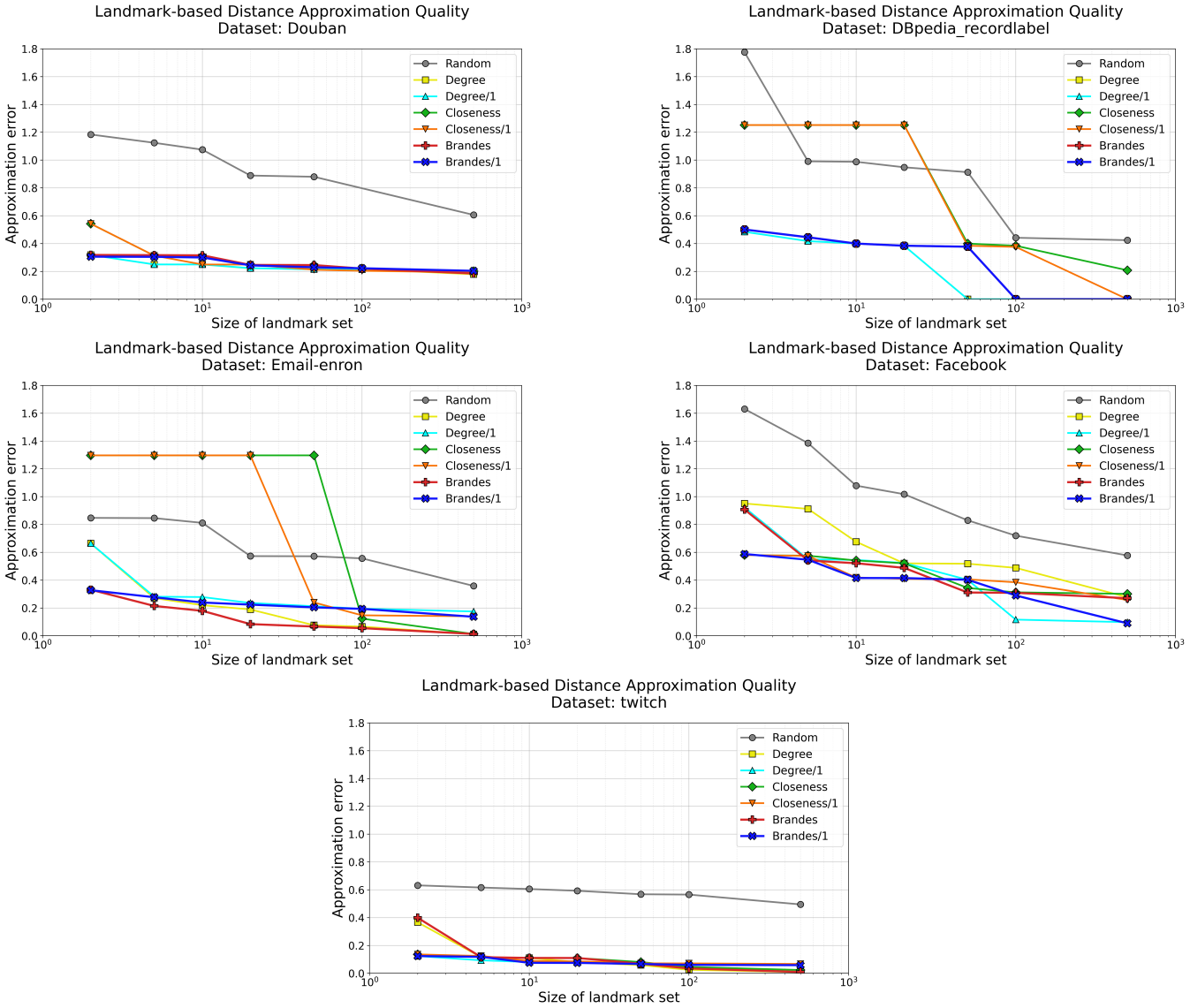


Figure 4: Approximation error of different landmark selection strategies on five real-world networks. Lower values indicate better performance.

across datasets. For example, Email-Enron benefits substantially from informed selection at very small budgets, whereas DBpedia shows more limited gains despite having a comparable clustering coefficient. This indicates that the impact of the first few landmarks is likely influenced by a combination of factors, including degree heterogeneity, bipartite or affiliation structure, and the spatial distribution of high-degree nodes, rather than by any single scalar graph property.

6.4 Computational Efficiency

The offline computation time depends on the strategy, and its components are summarised in Table 2 for $|D| = 100$ on the Facebook dataset. We use the Facebook Page-Page dataset as a representative benchmark to illustrate the relative computational costs and scaling behaviour of the different landmark selection strategies.

The closeness variants require a centrality-estimation phase based on 200 sampled BFS runs, which explains their higher selection cost. The Brandes-based strategies add a lightweight dependency-accumulation step that increases the running time by a few seconds but remains far cheaper than computing the exact betweenness. For Brandes, the distance-table construction additionally performs dependency accumulation during each BFS, which explains the higher T_{distance} values. Landmark selection is inexpensive for Random and Degree, while Sampled Closeness incurs extra cost due

to the sampled BFS runs used to estimate centrality. The distance-table construction dominates the total preprocessing time because it requires one BFS per landmark.

Table 2: Index construction time for $|D| = 100$ landmarks on the Facebook dataset.

Strategy	T_{LM} [s]	T_{distance} [s]	T_{index} [s]
Random	0.0002	1.99	1.99
Degree	0.25	1.99	2.25
Degree (hop-1)	0.26	2.13	2.39
Closeness	4.85	1.97	6.82
Closeness (hop-1)	4.82	2.05	6.87
Brandes	1.35	6.55	7.90
Brandes (hop-1)	1.41	6.63	8.04

T_{LM}: landmark-selection time, T_{distance}: distance-table construction time, T_{index}: total index construction-time ($T_{\text{LM}} + T_{\text{distance}}$).

Figure 5 shows how the total preprocessing time scales with the landmark budget on the Facebook dataset. The cost increases nearly linearly with $|D|$, since the distance table requires one BFS per landmark. Degree and Closeness scale similarly, while Brandes is higher because each BFS additionally accumulates dependencies.

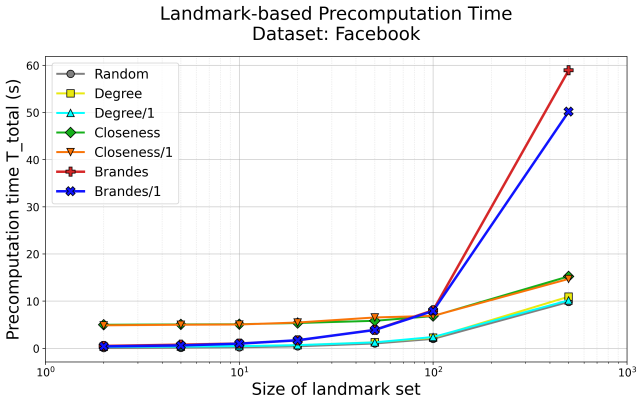


Figure 5: Total precomputation time as a function of the landmark budget $|D|$ on the Facebook dataset.

In the online phase, computing an estimate for a pair (u, v) requires combining their distances to the $|D|$ landmarks, so the theoretical cost per query is $O(|D|)$. Table 3 reports the average query time on the Facebook dataset for different landmark budgets. The latency grows slowly with $|D|$, in line with this linear dependence, and remains well below one millisecond even for $|D| = 500$. Compared to the offline preprocessing, the online computation is therefore negligible.

Table 3: Average distance query time for different landmark budgets on the Facebook dataset (averaged over all strategies).

$\lvert D\rvert$	T_{query} [ms]
2	0.0046
5	0.0056
10	0.0068
20	0.0121
50	0.0305
100	0.0633
500	0.68

$|D|$: number of landmarks; T_{query}: values averaged over all landmark strategies.

7 Conclusion

The effectiveness of landmark-based distance estimation in large graphs depends strongly on how the landmarks are chosen. In this report we compared three standard selection heuristics: Random, Degree, and Sampled Closeness, with a structure-aware strategy that uses a Brandes-inspired shortest-path participation signal, optionally combined with hop-exclusion.

Experiments on five real-world networks show that most of the differences between strategies occur when the landmark budget is small. Random selection is consistently the weakest baseline. Degree and the Brandes-inspired strategy usually provide the lowest errors at very small budgets, while Sampled Closeness becomes competitive once more landmarks are available. Hop-exclusion can improve the first few landmark choices on some datasets, but its effect is not uniform across graphs. In terms of efficiency, index construction grows roughly linearly with the number of landmarks because it is dominated by BFS runs for building the landmark distance tables; for Brandes, each BFS additionally performs dependency accumulation, which explains its higher construction time. Query answering remains fast, since it only scans the stored distances to the landmarks.

An interesting direction for future work is to better predict which graphs benefit from hop-exclusion and from participation-based selection, using measurable graph statistics. Another is to study incremental updates of landmarks and distance tables in evolving networks.

Acknowledgments

We would like to thank Professor Frank W. Takes and all the teaching assistants of the course for their guidance, feedback, and support throughout the project.

References

[1] [n. d.]. DBpedia Record Label Network. https://networks.skewed.de/net/dbpedia_recordlabel. Accessed: 2025-11-21.
[2] [n. d.]. Douban freindship network. https://networks.skewed.de/net/douban. Accessed: 2025-11-21.
[3] [n. d.]. email-Eu-core network. https://snap.stanford.edu/data/email-Enron.html. Accessed: 2025-11-21.
[4] [n. d.]. Facebook Large Page-Page network. https://snap.stanford.edu/data/facebook-large-page-page-network.html. Accessed: 2025-11-21.
[5] [n. d.]. Twitch Gamers Social Network. https://snap.stanford.edu/data/twitch_gamers.html. Accessed: 2025-11-21.

[6] Stephen P. Borgatti, Martin G. Everett, and Linton C. Freeman. 2005. Social network analysis: Analytical techniques. *Journal of Organizational Behavior* 26, 3 (2005), 315–344.
[7] Ulrik Brandes. 2001. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology* 25, 2 (2001), 163–177.
[8] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. *Introduction to Algorithms* (3rd ed.). MIT Press.
[9] E. W. Dijkstra. 1959. A note on two problems in connexion with graphs. *Numer. Math.* 1 (1959), 269–271.
[10] R. W. Floyd. 1962. Algorithm 97: Shortest path. *Commun. ACM* 5, 6 (1962), 345.
[11] Andrew V. Goldberg and Chris Harrelson. 2005. Computing the shortest path: A* meets ALT. In *Proceedings of the 8th International Symposium on Experimental Algorithms (SEA)*. 196–207.
[12] Piotr Indyk and Jiri Matousek. 1998. Low-distortion embeddings of finite metric spaces. In *Proceedings of the 29th ACM Symposium on Theory of Computing (STOC)*. 438–444.
[13] Michalis Potamias, Francesco Bonchi, Carlos Castillo, and Aristides Gionis. 2009. Fast shortest path distance estimation in large networks. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM)*. ACM, Hong Kong, China, 867–876.
[14] Christian Sommer. 2014. Shortest-path queries in static networks. *Comput. Surveys* 46, 4 (2014), 45.
[15] Mikkel Thorup and Uri Zwick. 2005. Approximate distance oracles. In *Proceedings of the 35th Annual ACM Symposium on Theory of Computing (STOC)*. 183–192.